Klausur / Aufgabenkatalog: Coding und Digitalisierung

Technische Hochschule Köln — (Teil A)

Hinweise für Studierende (bzw. Auswahlkatalog für Lehrende):

- **Bearbeitungszeit (Zielklausur):** 45 Minuten (Auswahl aus diesen 40 Aufgaben).
- **Hilfsmittel:** Keine elektronischen Hilfsmittel (Handys, Laptops, Smartwatches etc.).
- **Antwortformat:** Antworten Sie kurz, präzise und technisch exakt direkt unter den Aufgaben.
- **Achtung: Dies sind viel mehr Aufgaben, als in der richtigen Klausur!!**

Nutzen Sie diese Aufagen als Übungskript. Versuchen Sie zunächst die Aufgaben selber zu lösen. Vergleichen Sie Ihre Antworten jeweils mit den Antworen von z.B. dem KI Chat der TH Köln. Fragen Sie nicht nach der Lösung, immer nach der Erklärung.

Teil 1: Python-Architektur, Typisierung, Operatoren & Kontrollfluss (Aufgaben 1–10)

1. **Python-Ausführungsmodell und Bytecode-Sicherheit (4 Pkt.)**
   Ein Ingenieur behauptet: *„Da Python den Quellcode vor der Ausführung in Bytecode (`.pyc`) übersetzt, ist unser proprietärer Regelungsalgorithmus vor neugierigen Kunden sicher geschützt und läuft direkt auf der CPU."* Widerlegen Sie beide Teile dieser Aussage technisch präzise anhand der Funktionsweise der Python Virtual Machine (PVM).

2. **Virtuelle Umgebungen (venv) in der Produktion (3 Pkt.)**
   Auf einem Industrie-PC laufen zwei Python-Skripte: Projekt A benötigt `pandas 1.5`, Projekt B benötigt zwingend `pandas 2.2` (in welchem alte Methoden entfernt wurden). Was passiert ohne `venv` bei einer globalen Installation mit `pip`, und wie löst `python -m venv .venv` dieses Problem auf Dateisystemebene?

3. **Operator-Präzedenz und Typ-Promotion (4 Pkt.)**
   Geben Sie den exakten Wert sowie den resultierenden Datentyp (`int`, `float`, `bool`) für jede Variable nach der Ausführung an:

```
a = 17 // 5 + 17 % 5 ** 1
b = 4 * 2.0 == 8 and not (10 % 3 == 0)
c = 2 ** 3 ** 2
```

   `a =` _____ (Typ: _____) `b =` _____ (Typ: _____) `c =` _____

4. **Gleitkomma-Arithmetik (IEEE 754) und Sicherheitsrisiken (4 Pkt.)**
   Betrachten Sie folgende Abschaltbedingung eines Druckventils:

```
druck = 0.1 + 0.2
if druck == 0.3:
    print("Ventil␣schliessen")
else:
    print("Weiterpumpen!")
```

   Welche Ausgabe erzeugt dieser Code tatsächlich? Warum ist der Vergleich mit == bei `float`-Werten in Steuerungen gefährlich, und wie prüft man dies mathematisch sauber?

5. **Logischer Fehler in Verzweigungs-Kaskaden (if / elif / else) (5 Pkt.)**
   Der folgende Code soll einen Sensorwert kategorisieren. Finden Sie den gravierenden logischen Fehler:

```
o2 = 0.2
if o2 > 0.0:
    status = "Normalbetrieb"
elif o2 < 0.5:
    status = "KRITISCHER␣SAUERSTOFFMANGEL!"
else:
    status = "Sensorfehler␣(Negativ)"
print(status)
```

   Welchen Wert hat `status` am Ende? Warum wird der kritische Alarm bei `o2 = 0.2` niemals ausgelöst, und wie muss die Bedingung korrigiert werden?

6. **Kurzschlussauswertung (Short-Circuit Evaluation) (4 Pkt.)**
   Stürzt der folgende Code mit einem `ZeroDivisionError` ab oder läuft er fehlerfrei durch? Begründen Sie Ihre Antwort anhand der Auswertungsreihenfolge von `and` und `or`.

```
anzahl = 0
summe = 15.0
if anzahl > 0 and (summe / anzahl) > 5.0:
    print("Hoher␣Schnitt")
else:
    print("Keine␣Daten␣oder␣niedriger␣Schnitt")
```

7. **Komplexes Loop-Tracing mit `range`, `continue` und `break` (5 Pkt.)**
   Verfolgen Sie die Schleife Schritt für Schritt. Welche exakte Ausgabe erscheint im Terminal?

```
total = 0
for i in range(1, 12, 2):
    if i % 3 == 0:
        continue
    if total > 6:
        break
    total += i
    print(f"i={i},␣total={total}")
```

8. **Endlosschleife (Deadlock) in einer Heizungssteuerung analysieren (4 Pkt.)**
   Warum friert die folgende Steuerung in einer Endlosschleife ein, und welche einzelne Codezeile fehlt bzw. ist falsch eingerückt?

```
temp = 16.0
ziel = 21.0
schritt = 1.5
while temp < ziel:
    if temp == 19.0:
        print("Schwellenwert␣19␣Grad␣passiert")
        continue
    temp += schritt
```

9. **Verschachtelte Schleifen: Laufzeit und Zählwert (4 Pkt.)**
   Welchen exakten Zahlenwert gibt `print(zaehler)` am Ende aus?

```
zaehler = 0
matrix = [[1, 2, 3], [4, -1, 6], [7, 8, 9]]
for zeile in matrix:
    for wert in zeile:
        if wert < 0:
            break
        zaehler += 1
print(zaehler)
```

10. **f-Strings und Formatierungsspezifizierer (3 Pkt.)**
    Gegeben sind `sensor = "BB1"` und `messwert = 2.1468`. Welche exakte Zeichenkette gibt der Befehl `print(f"[sensor] O2: {messwert:.2f} mg/L (Gerundet: {round(messwert, 1)})")` aus?

Teil 2: Komplexe Datenstrukturen & Algorithmik (Aufgaben 11–18)

11. **Aliasing vs. Slicing (Speicherreferenzen bei Listen) (5 Pkt.)**
    Geben Sie die exakte Ausgabe der beiden `print`-Befehle an und erklären Sie den Unterschied zwischen der Zuweisung `b = a` und `c = a[:]`.

```
a = [10, 20, 30]
b = a
c = a[:]
b.append(40)
c[0] = 99
print("a:", a)
print("c:", c)
```

12. **Gefährlicher Anti-Pattern: Liste während der Iteration verändern (5 Pkt.)**
    Ein Programmierer möchte alle negativen Fehlerwerte aus einer Liste löschen:

```
werte = [4.2, -1.0, -2.0, 4.5]
for w in werte:
    if w < 0:
        werte.remove(w)
print(werte)
```

Warum bleibt `-2.0` trotz der Schleife in der Liste `werte` enthalten? Geben Sie die Ausgabe an und erklären Sie das Index-Verschiebungsproblem.

13. **Tupel-Immutabilität und verschachtelte Mutabilität (4 Pkt.)**
Ist der folgende Code ausführbar oder wirft er einen `TypeError`? Begründen Sie genau!

```
sensor_paket = (101, "O2", [2.1, 2.3])
sensor_paket[2].append(2.5)
print(sensor_paket)
```

14. **Dictionary-Schlüssel und Hashability (4 Pkt.)**
Welche der folgenden drei Zeilen führt zu einem Laufzeitfehler (`TypeError: unhashable type`) und warum?

```
d = {}
d["standort_1"] = 22.5          # Zeile A
d[(50.93, 6.96)] = 18.2         # Zeile B (Koordinaten-Tupel)
d[[50.93, 6.96]] = 19.0         # Zeile C (Koordinaten-Liste)
```

15. **Sichere Dictionary-Abfragen (`KeyError` vermeiden) (4 Pkt.)**
Gegeben ist `messung = {ïd: 101, ö2: 4.2}`. Was passiert beim Aufruf `print(messung[ttemp"])`, und wie können Sie den Wert mit einer eingebauten Dictionary-Methode abrufen, sodass bei fehlendem Schlüssel automatisch `-99.9` statt eines Absturzes geliefert wird?

16. **Aggregation über eine Liste von Dictionaries (Code schreiben) (6 Pkt.)**
Gegeben ist eine Liste von Sensor-Dictionaries:

```
sensoren = [
    {"id": 1, "aktiv": True,  "wert": 4.0},
    {"id": 2, "aktiv": False, "wert": 99.9},
    {"id": 3, "aktiv": True,  "wert": 2.0}
]
```

Schreiben Sie eine `for`-Schleife, die den Mittelwert aller Schlüssel `"wert"` berechnet, aber **nur** für Sensoren, bei denen `äktiv"` den Wert `True` hat.

17. **Algorithmik: Klassische Median-Berechnung in Standard-Python (5 Pkt.)**
Ergänzen Sie die fehlende Index-Logik für eine gerade Anzahl an Elementen in der sortierten Liste:

```
daten = [20, 10, 40, 30]
s = sorted(daten)
n = len(s)
if n % 2 == 0:
    # Berechnen Sie hier den korrekten Median fuer gerade Listenlaengen
        :
    median = ____________________________________________________
else:
    median = s[n // 2]
```

Welchen Wert hat `median` für die Liste `[20, 10, 40, 30]`? _____

18. **Ressourcen-Management: Datei-Import mit `with open()` (4 Pkt.)**
Warum ist das Einlesen einer CSV-Datei mittels `with open("daten.csv", rr") as f:` einem einfachen `f = open("daten.csv", rr")` gefolgt von `f.close()` am Ende des Skripts in produktiven Systemen zwingend vorzuziehen? Was passiert, wenn während der Verarbeitung in Zeile 500 eine Exception auftritt?

Teil 3: Funktionen, Clean Code, Scope & Exceptions (Aufgaben 19–26)

19. **WET vs. DRY und versteckte Copy-Paste-Bugs (4 Pkt.)**
Analysieren Sie diesen historischen Code-Ausschnitt:

```
b1 = [4.0, 4.0]
b2 = [2.0, 2.0, 2.0, 2.0]
mw1 = sum(b1) / len(b1)
mw2 = sum(b2) / len(b1)
```

Welchen falschen Zahlenwert liefert `mw2`? Warum ist dieser Fehlertyp in einer industriellen Steuerung gefährlicher als ein Syntaxfehler?

20. **Die `print()`-vs.-`return`-Falle (`NoneType`-Crash) (5 Pkt.)**
Ein Studierender schreibt folgende Funktion und ruft sie im Hauptprogramm auf:

```
def berechne_offset(x):
    erg = x + 1.5
    print(erg)

kalibriert = berechne_offset(4.0) * 2.0
```

Das Programm stürzt in der letzten Zeile mit `TypeError: unsupported operand type(s) for *: 'NoneType' and 'float'` ab. Erklären Sie präzise, warum dieser Fehler trotz des `print(erg)` entsteht, und korrigieren Sie die Funktion.

21. **Default-Parameter und Syntaxregeln im Funktionskopf (4 Pkt.)**
Warum lässt sich die folgende Funktionsdefinition in Python **nicht** ausführen (`SyntaxError`), und wie muss der Funktionskopf korrekt lauten?

```
def filtere_daten(min_wert=0.0, daten_liste, max_wert=50.0):
    return [x for x in daten_liste if min_wert <= x <= max_wert]
```

22. **Tuple Unpacking bei mehrfachen Rückgabewerten (4 Pkt.)**
Welche exakte Ausgabe erzeugt der folgende Code? Welchen Datentyp hat die Variable `paket`?

```
def statistik(werte):
    return min(werte), max(werte), max(werte) - min(werte)

paket = statistik([2.0, 5.5, 1.5, 4.0])
low, high, span = paket
```

```
print(f"Typ: {type(paket).__name__} | Spanne: {span} | Max: {paket[1]}"
    )
```

23. **Gültigkeitsbereich (Scope):** `UnboundLocalError` **verstehen (5 Pkt.)**
Warum wirft die folgende Funktion beim Aufruf `erhoehe_zaehler()` einen `UnboundLocalError`, obwohl `fehler_anzahl` außerhalb definiert wurde? Wie löst man dies im Sinne von Clean Code (ohne das Schlüsselwort `global`)?

```
fehler_anzahl = 0

def erhoehe_zaehler():
    fehler_anzahl += 1
    return fehler_anzahl
```

24. **Ausnahmebehandlung: Warum ein „nacktes“ `except:` gefährlich ist (4 Pkt.)**
Vergleichen Sie `except ValueError:` mit einem pauschalen `except:` (ohne Angabe eines Fehlertyps). Warum gilt ein nacktes `except:` in der Softwareentwicklung als schwerer Designfehler? Geben Sie ein konkretes Beispiel für einen Programmierfehler im `try`-Block, der dadurch unbemerkt verschluckt würde.

25. **Ablaufsteuerung in `try / except`-Blöcken (5 Pkt.)**
Geben Sie Zeile für Zeile die exakte Konsolenausgabe des folgenden Programms an:

```
daten = ["10.0", "Fehler", "0.0"]
for d in daten:
    try:
        w = float(d)
        q = 100.0 / w
        print(f"Erfolg: {q}")
    except ValueError:
        print("Typfehler erkannt")
    except ZeroDivisionError:
        print("Division durch Null erkannt")
    print("--- Schritt Ende ---")
```

26. **Code schreiben: Robuster Diagnose-Parser mit Fehler-Logging (8 Pkt.)**
Vervollständigen Sie die Funktion `parse_stream(roh_liste, max_grenze)`. Sie soll gültige Strings in `float` umwandeln und prüfen, ob diese `<= max_grenze` sind. Ungültige Texte (`ValueError`) und Grenzwertüberschreitungen sollen gezählt werden. Die Funktion gibt am Ende das Tupel `(saubere_liste, fehler_anzahl)` zurück.

```
def parse_stream(roh_liste, max_grenze=50.0):
    sauber = []
    fehler = 0
    for eintrag in roh_liste:
        # --- Ihr Code ab hier ---









        # --- Ende Ihres Codes ---
    return sauber, fehler
```

Teil 4: Vektorisierte Datenanalyse mit NumPy (Aufgaben 27–31)

27. **Python-Liste vs. NumPy-Array: Operator-Überladung (4 Pkt.)**
Geben Sie die exakte Ausgabe für beide `print`-Anweisungen an:

```
import numpy as np
liste = [1, 2, 3]
arr = np.array([1, 2, 3])
print(liste * 2)
print(arr * 2 + 10)
```

28. **Homogenitätszwang (Type Coercion) in `numpy.ndarray` (4 Pkt.)**
Ein Entwickler übergibt einen ungefilterten Sensor-Stream direkt an NumPy:

```
import numpy as np
arr = np.array([2.14, 2.08, "Error", 1.95])
```

Warum schlägt eine anschließende Rechnung `arr * 2` fehl, selbst auf den ersten beiden Elementen? Welchen Datentyp (`dtype`) hat NumPy für `arr` automatisch gewählt und warum?

29. **Syntaxfehler bei kombinierter Boolescher Maskierung in NumPy (5 Pkt.)**
Der folgende Code zum Filtern eines Arrays `temp` stürzt sofort mit einem `ValueError` bzw. `TypeError` ab. Finden Sie die **zwei** Fehler in Zeile 2 und schreiben Sie die Zeile korrekt auf:

```
temp = np.array([11.4, -5.0, 15.2, 105.0, 12.0])
maske = temp >= 0.0 and temp <= 40.0
```

Korrekte Zeile 2: `maske =` _____

30. **Vektorisierte Regelabweichung und `np.where()` (6 Pkt.)**
Gegeben sind zwei Arrays für Soll- und Ist-Werte des Sauerstoffs:

```
import numpy as np
soll = np.array([2.0, 2.0, 2.5, 2.5])
ist  = np.array([2.2, 1.5, 1.0, 2.5])
defizit = soll - ist
geblaese_kw = np.where(defizit > 0, 10.0 + 20.0 * defizit, 10.0)
print("Defizit:", defizit)
print("Leistung:", geblaese_kw)
```

Berechnen Sie die exakten Werte der beiden Arrays `defizit` und `geblaese_kw`, die ausgegeben werden.

31. **Statistische Robustheit: Mittelwert (`np.mean`) vs. Median (`np.median`) (4 Pkt.)**
In einem Array `nh4 = np.array([2.0, 2.1, 1.9, 2.0, 999.0])` befindet sich ein einzelner extremer Sensor-Ausreißer (`999.0`) durch Überspannung. Berechnen Sie im Kopf (ca.) den Mittelwert und exakt den Median dieses Arrays. Welche Kennzahl ist robuster und warum?

Teil 5: Pandas DataFrames & Zeitreihen-Pipelines (Aufgaben 32–36)

32. **Fallstricke beim CSV-Import deutscher Industriedaten (`pd.read_csv`) (4 Pkt.)**
Sie exportieren eine CSV-Datei aus einer deutschen SPS (Trennzeichen ist Semikolon `;`, Dezimalzeichen ist Komma `,`, z. B. `08:00;2,14;11,4`). Was passiert, wenn Sie diese Datei nur mit dem Standardbefehl `df = pd.read_csv(ßps.csv")` laden, und wie lautet der exakt korrekte Funktionsaufruf mit Parametern?

33. **Diagnose mit `df.info()` und `df.describe()` (5 Pkt.)**
Die Ausgabe von `df.info()` für einen Datensatz mit `RangeIndex: 1000 entries` zeigt: `0 zeit 1000 non-null object`
`1 o2_ist 940 non-null float64`
`2 nh4_mg_l 1000 non-null float64`
Beantworten Sie kurz:

    (a) Wie viele Messwerte fehlen (`NaN`) in der Spalte `o2_ist`? _____

    (b) Warum erscheint die Spalte `zeit` nicht in der Statistik-Tabelle von `df.describe()`?

    (c) Mit welchem Pandas-Befehl zählen Sie die Anzahl der `NaN`-Werte für alle Spalten direkt im Code aus?

34. **Datenbereinigungs-Strategien im Vergleich (`fillna`, `ffill`, `interpolate`) (6 Pkt.)**
Gegeben ist eine Pandas-Series mit einer Datenlücke: `s = pd.Series([10.0, np.nan, 20.0])`. Welche konkreten 3 Zahlenwerte enthält die Series jeweils nach Ausführung von:

    (a) `s.fillna(0.0)` → _____

    (b) `s.ffill()` → _____

    (c) `s.interpolate()` → _____

    Warum ist `fillna(0.0)` bei Wassertemperatur-Zeitreihen ein schwerer methodischer Fehler, und unter welcher Bedingung bleibt selbst nach `s.ffill()` noch ein `NaN` in der Spalte übrig?

35. **Boolean Indexing und der `dropna()`-Seiteneffekt (6 Pkt.)**
In einem DataFrame `df` (20 Zeilen) soll ein einzelner negativer Kalibrierfehler (`-0.5`) in der Spalte `nh4` entfernt werden. In derselben Spalte stehen aber auch zwei wichtige Zeilen mit `NaN`, die später interpoliert werden sollen. Warum löscht der Befehl `df = df[df['nh4'] >= 0.0]` ungewollt auch die beiden `NaN`-Zeilen (sodass nur 17 statt 19 Zeilen bleiben), und wie muss die Filterbedingung mit `.isna()` ergänzt werden?

36. **Zeitreihen-Synchronisation und Resampling (Code-Analyse & Rechnung) (7 Pkt.)**
Gegeben ist folgender Code für asynchrone Sensordaten:

```
import pandas as pd
df = pd.DataFrame({
```

```
    'zeit': ['2024-01-01␣08:01', '2024-01-01␣08:04', '2024-01-01␣08:11'
       ],
    'o2':   [2.0, 2.6, 3.5]
})
# Schritt A: Zeitspalte konvertieren und als Index setzen
df['zeit'] = pd.to_datetime(df['zeit'])
df.set_index('zeit', inplace=True)

# Schritt B: Resampling
df_5m = df.resample('5min').mean()
```

(a) Welche Fehlermeldung tritt in Schritt B auf, wenn man Schritt A vergisst?

(b) Notieren Sie alle Zeitstempel (Index) und die zugehörigen `o2`-Werte von `df_5m` nach Schritt B (Achtung: Was passiert mit dem Intervall `08:05:00`, in dem keine Messung lag?).

Teil 6: Objektorientierte Programmierung (OOP) & Synthese (Aufgaben 37–40)

37. **OOP-Konzepte, Konstruktor `__init__` und die Rolle von `self` (5 Pkt.)**
Ein Studierender schreibt folgende Klasse:

```
class Sensor:
    def __init__(self, s_id):
        id = s_id
        self.historie = []

s1 = Sensor(101)
print(s1.id)
```

Das Programm bricht in der letzten Zeile mit `AttributeError: 'Sensor' object has no attribute 'id'` ab. Erklären Sie genau, warum dieser Fehler auftritt und welche fundamentale Aufgabe die Referenz `self` beim Erzeugen mehrerer Sensor-Instanzen (`s1`, `s2`) übernimmt.

38. **Schwerwiegender Bug: Klassenattribut vs. Instanzattribut (6 Pkt.)**
Betrachten Sie das folgende Klassendesign und die Ausgabe:

```
class Messstelle:
    daten = []  # <-- Achtung!
    def __init__(self, name):
        self.name = name
    def add(self, w):
        self.daten.append(w)

m1 = Messstelle("Zulauf")
m2 = Messstelle("Ablauf")
m1.add(4.2)
m2.add(1.8)
print(m1.daten)
```

Welche Ausgabe liefert `print(m1.daten)`? Warum teilen sich `m1` und `m2` hier dieselbe Liste, und wie muss der Code korrigiert werden, damit jede Instanz ihre eigene Liste besitzt?

39. **Zustandsautomaten und Kapselung in Klassen (`self.ist_aufbereitet`) (6 Pkt.)**
In der Vorlesung wurde in der Klasse `BeckenAnalyse` ein boolesches Attribut `self.ist_aufbereitet = False` im Konstruktor angelegt, welches erst am Ende der Methode `aufbereiten_5min(self)` auf `True` gesetzt wird.

(a) Welcher Laufzeitfehler würde in der Methode `zeige_report(self)` beim Zugriff auf `self.df_5min['o2'].mean()` auftreten, wenn ein Nutzer im Hauptprogramm `zeige_report()` vor `aufbereiten_5min()` aufruft (solange `self.df_5min = None` ist)?

(b) Schreiben Sie die 3 Zeilen Schutz-Code (Guard Clause) für den Anfang von `zeige_report(self)`, die diesen Absturz sauber abfangen.

40. **Große Synthese-Aufgabe: Fehlertolerante Klasse `SmartSensor` implementieren (15 Pkt.)**
Implementieren Sie eine vollständige Python-Klasse `SmartSensor`, welche die Konzepte aus OOP, `try/except`, Grenzwertfilterung und Statistik vereint:

    - **Konstruktor `__init__(self, sensor_id, min_w=0.0, max_w=50.0)`:** Speichert die drei Parameter als Objektattribute und legt eine leere Liste `self.werte = []` sowie einen Zähler `self.fehler = 0` an.
    - **Methode `lese_wert(self, roh_wert)`:** Versucht `roh_wert` mittels `float()` umzuwandeln.
        - Schlägt dies mit `ValueError` oder `TypeError` fehl, wird `self.fehler` um 1 erhöht und `False` zurückgegeben.
        - Gelingt die Umwandlung, wird geprüft, ob der Wert zwischen `self.min_w` und `self.max_w` (jeweils inklusive) liegt. Falls ja, wird er an `self.werte` angehängt und `True` zurückgegeben; falls nein (Ausreißer), wird `self.fehler` um 1 erhöht und `False` zurückgegeben.
    - **Methode `get_kennzahlen(self)`:** Gibt ein Dictionary `{änzahl: ..., mmittelwert: ..., ffehler: ...}` zurück. Ist `self.werte` noch leer, soll `mmittelwert"` den Wert `0.0` haben (Division durch Null verhindern!).

```
class SmartSensor:
    # --- Schreiben Sie Ihren vollstaendigen Klassen-Code hier: ---









# --- Kurzer Aufruf-Test (2 Zeilen): Instanz mit ID 101 anlegen und
    "-5.0" einlesen ---
```

— Ende des Aufgabenkatalogs (40 Prüfungsaufgaben) —